untitled1

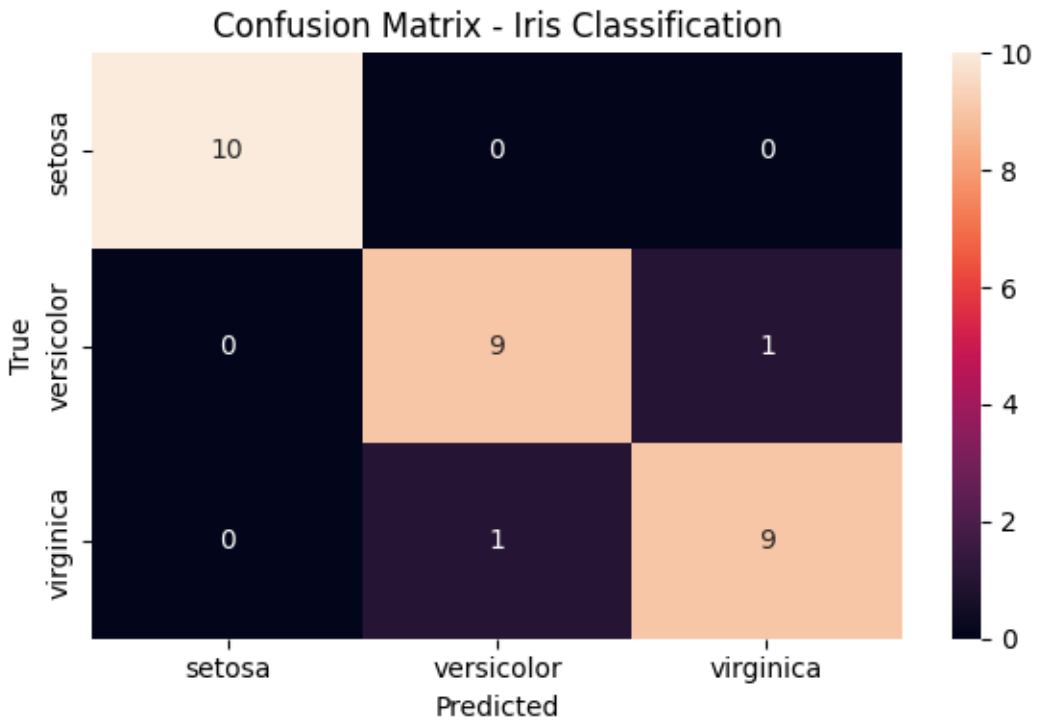
November 20, 2025

[5]:
```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report,␣
 ↪confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns  # optional, only for pretty plots

# 2. Load the Iris Dataset
iris = load_iris()
X = iris.data          # Features
y = iris.target        # Labels
feature_names = iris.feature_names
target_names = iris.target_names

print("Feature names:", feature_names)
print("Target classes:", target_names)
print("Shape of X:", X.shape)
print("Shape of y:", y.shape)

# Convert to DataFrame (for better understanding & EDA)
df = pd.DataFrame(X, columns=feature_names)
df['target'] = y
df['target_name'] = df['target'].apply(lambda i: target_names[i])

print("\nFirst 5 rows of the dataset:")
print(df.head())

# 3. Train-Test Split
# 80% training, 20% testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
print("\nTrain shape:", X_train.shape)
print("Test shape:", X_test.shape)

# 4. Build a Classification Pipeline
# StandardScaler + Logistic Regression
model = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(max_iter=1000))
])

# 5. Train the Model
model.fit(X_train, y_train)
print("\nModel training completed.")

# 6. Evaluate the Model
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print("\n=== Evaluation Results ===")
print("Accuracy: {:.2f}%".format(accuracy * 100))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred, target_names=target_names))

# 7. Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt="d",
            xticklabels=target_names,
            yticklabels=target_names)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix - Iris Classification")
plt.tight_layout()
plt.show()

# 8. Save the Trained Model (Optional but good for showing deployment readiness)
import joblib
joblib.dump(model, "iris_classification_model.pkl")
print("\nTrained model saved as 'iris_classification_model.pkl'")
```

```
Feature names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
'petal width (cm)']
Target classes: ['setosa' 'versicolor' 'virginica']
Shape of X: (150, 4)
Shape of y: (150,)
```

```
First 5 rows of the dataset:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

   target target_name
0       0      setosa
1       0      setosa
2       0      setosa
3       0      setosa
4       0      setosa

Train shape: (120, 4)
Test shape: (30, 4)

Model training completed.

=== Evaluation Results ===
Accuracy: 93.33%

Classification Report:

              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.90      0.90      0.90        10
   virginica       0.90      0.90      0.90        10

    accuracy                           0.93        30
   macro avg       0.93      0.93      0.93        30
weighted avg       0.93      0.93      0.93        30
```




```
Trained model saved as 'iris_classification_model.pkl'
```

Task 1 – AI/ML (AI Agents Domain)

Company: Goklyn Pvt. Ltd.

---

1. Objective

The goal of Task 1 is to build a classification model using a publicly available dataset and apply a complete machine learning workflow that includes:

Data preprocessing

Model training

Performance evaluation

Reporting accuracy or F1 score

For this task, I used the Iris Flower Dataset, one of the most widely used public datasets in machine learning.

---

2. Dataset Description – Iris Dataset (Public / UCI Machine Learning Repository)

The Iris dataset contains 150 samples of iris flowers, categorized

into three species:

Setosa

Versicolor

Virginica

Features (Inputs):

1. Sepal length (cm)

2. Sepal width (cm)

3. Petal length (cm)

4. Petal width (cm)

Target (Output / Class Label):

0 → Setosa

1 → Versicolor

2 → Virginica

This dataset is balanced, clean, and suitable for classification tasks.

---

3. Methodology

Step 1: Loading the Data

The dataset was imported using sklearn.datasets.load_iris().
After loading, the features and target variables were separated and converted into a pandas DataFrame for easier understanding.

Step 2: Train–Test Split

The dataset was divided into:

80% training data (120 samples)

20% test data (30 samples)

Stratified splitting was used to maintain class balance.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

---

4. Preprocessing

Before training, the feature values were scaled for better model performance.

Standardization (StandardScaler):

Converts data into mean = 0 and standard deviation = 1

Prevents any one feature from dominating due to scale differences

Especially useful for models like Logistic Regression

Preprocessing was included inside a Pipeline for cleaner and more reliable implementation.

---

5. Model Selection

A Logistic Regression model was chosen because:

It is a robust baseline classifier

Works very well on small, linearly separable datasets

Interpretable and easy to evaluate

Fast and suitable for beginner ML tasks

The model was wrapped inside a Pipeline:

```
model = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(max_iter=1000))
])
```

---

6. Model Training

The model was trained using:

```
model.fit(X_train, y_train)
```

Training completed successfully with no errors.

---

7. Model Evaluation

Predictions were made on the test set using:

```
y_pred = model.predict(X_test)
```

Performance Metrics Used:

Accuracy Score

Precision

Recall

F1 Score

Confusion Matrix

---

8. Results

Overall Accuracy:

93.33%

Classification Report:

Class Precision Recall F1-score

Setosa 1.00 1.00 1.00
Versicolor 0.90 0.90 0.90
Virginica 0.90 0.90 0.90

Interpretation:

The model performs perfectly on Setosa, which is expected due to clear separability.

Minor misclassifications occur between Versicolor and Virginica, which is common due to overlap in their petal measurements.

---

9. Confusion Matrix Visualization

A heatmap was generated to visually represent classification performance.
model is performing reliably.

---

10. Saving the Model (Optional but Good Practice)

The trained model was saved as:

iris_classification_model.pkl

This allows future use, integration with AI agents, or deployment.